

Thực hành Phần 2 — tổng quan

- 6 bài lab bám sát 6 khối lý thuyết vừa học: (1) tính toán replication, (2) đọc code HDFS thật, (3) mô phỏng rack-aware placement, (4) so sánh CSV vs Parquet, (5) khám phá Delta Lake transaction log, (6) so sánh table format
- Toàn bộ dữ liệu/code mẫu sử dụng lại các repo đã tải về trong `TaiLieuThamKhao/P2_LuuTruPhanTan/`: `hadoop-book-code`, `delta-lake-examples`, `iceberg-spark-docker-quickstart`
- Kết thúc bằng bài tập thiết kế nhóm (mở rộng case study đã học) và checklist chuẩn bị cho Tuần 4 (MapReduce/Spark)

Lab 1 — Bài toán: tính chi phí lưu trữ thực tế

- Một công ty cần lưu **500TB** dữ liệu log trên HDFS với replication factor mặc định = 3
- Câu hỏi 1: dung lượng vật lý thực tế cần là bao nhiêu?
- Câu hỏi 2: nếu chuyển 80% dữ liệu **cũ, ít truy cập** sang chính sách Erasure Coding (6 data + 3 parity, overhead $\approx 50\%$), tổng dung lượng vật lý mới là bao nhiêu? Tiết kiệm được bao nhiêu % so với replication toàn bộ?

Câu hỏi 1: $500\text{TB} \times 3 = 1.500\text{TB}$ dung lượng vật lý.

Câu hỏi 2: chia dữ liệu thành 2 phần:

$$20\% \text{ dữ liệu "nóng"} : 100\text{TB} \times 3 = 300\text{TB}$$

$$80\% \text{ dữ liệu "lạnh"} \text{ (Erasure Coding)} : 400\text{TB} \times 1.5 = 600\text{TB}$$

$$\text{Tổng mới} = 300 + 600 = 900\text{TB}$$

Tiết kiệm: $(1500 - 900)/1500 = 40\%$ dung lượng vật lý — minh họa vì sao các tổ chức lớn kết hợp cả hai chính sách thay vì chỉ dùng một loại duy nhất cho toàn bộ dữ liệu.

Lab 2 — Đọc code HDFS Java API thật (hadoop-book-code)

- Mở thư mục
TaiLieuThamKhao/P2_LuuTruPhanTan/hadoop-book-code/ch03-hdfs/ —
đây là code mẫu chính thức đi kèm sách *Hadoop: The Definitive Guide*
- Nhiệm vụ: tìm class đọc/ghi file trên HDFS bằng FileSystem API (ví dụ
URLCat.java, FileSystemCat.java), xác định: dòng code nào lấy **cấu hình cụm**
(Configuration), dòng nào gọi **FileSystem.get()** để kết nối NameNode
- So sánh với kiến trúc đã học: **FileSystem.get()** thực chất đang liên lạc với
NameNode để lấy thông tin, trước khi dữ liệu thực sự được đọc/ghi **trực tiếp với**
DataNode

Lab 2 — trích đoạn FileSystemCat.java (rút gọn)

```
// Nguồn: hadoop-book-code/ch03-hdfs (Tom White)
public class FileSystemCat {
    public static void main(String[] args) throws Exception {
        String uri = args[0];
        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(URI.create(uri), conf);
        // ^ liên lạc NameNode để lấy metadata/vị trí block
        InputStream in = null;
        try {
            in = fs.open(new Path(uri));
            // ^ từ đây đọc dữ liệu TRỰC TIẾP từ DataNode
            IOUtils.copyBytes(in, System.out, 4096, false);
        } finally {
            IOUtils.closeStream(in);
        }
    }
}
```

Lab 3 — Mô phỏng Rack-Aware Placement bằng Python

- Viết hàm chọn 3 DataNode để đặt 3 bản sao của một block, theo đúng chính sách đã học: 1 tại node ghi gốc, 1 ở rack khác, 1 ở node khác cùng rack đó
- Input: danh sách node kèm rack (ví dụ `{"node1": "rack1", "node2": "rack1", ...}`) và node client đang ghi
- Output: danh sách 3 node được chọn, kèm rack tương ứng để kiểm tra chính sách có được tuân thủ

Lab 3 — code Python mô phỏng

```
import random
nodes = { # node: rack
    "n1": "r1", "n2": "r1", "n3": "r1",
    "n4": "r2", "n5": "r2", "n6": "r2",
    "n7": "r3", "n8": "r3",
}
def place_replicas(client_node, nodes, seed=1):
    random.seed(seed)
    r1 = client_node
    rack1 = nodes[r1]
    other_racks = [n for n, r in nodes.items() if r != rack1]
    r2 = random.choice(other_racks)
    rack2 = nodes[r2]
    same_rack2 = [n for n, r in nodes.items() if r == rack2 and n != r2]
    r3 = random.choice(same_rack2)
    return [(r1, rack1), (r2, rack2), (r3, rack2)]

for rep in place_replicas("n1", nodes):
    print(rep)
# -> ('n1', 'r1'), ('n5', 'r2'), ('n4', 'r2') (ví dụ)
```

- Nếu cụm chỉ có **đúng 1 rack** (mọi node cùng rack), hàm trên sẽ báo lỗi ở bước nào? Vì sao đây là một tình huống **thực sự nguy hiểm** về mặt chịu lỗi?
- Nếu tăng replication factor lên 5 thay vì 3, cần sửa hàm như thế nào để vẫn tuân thủ nguyên tắc “không quá 2 bản sao trong cùng 1 rack”?

Lab 4 — So sánh thực nghiệm CSV vs. Parquet

- Tạo một DataFrame giả lập **200.000 dòng** dữ liệu giao dịch (mã đơn hàng, khu vực, giá trị, số lượng, ngày) bằng pandas + numpy
- Ghi ra cả hai định dạng: .csv và .parquet (nén Snappy) — so sánh **dung lượng file**
- Đo thời gian đọc **chỉ 1 cột** (gia_tri) từ mỗi định dạng — so sánh **tốc độ**
- Yêu cầu cài pyarrow: `pip install pandas pyarrow`

Lab 4 — code Python

```
import pandas as pd, numpy as np, os, time
np.random.seed(42)
n = 200_000
df = pd.DataFrame({
    "id": np.arange(n),
    "khu_vuc": np.random.choice(["HCM", "HN", "DN", "CT", "HP"], n),
    "gia_tri": np.random.exponential(50000, n).round(2),
    "so_luong": np.random.randint(1, 20, n),
    "ngay": pd.to_datetime("2026-01-01") +
            pd.to_timedelta(np.random.randint(0, 240, n), unit="D"),
})
df.to_csv("bench.csv", index=False)
df.to_parquet("bench.parquet", engine="pyarrow", compression="snappy")

print("CSV:", os.path.getsize("bench.csv")/1e6, "MB")
print("Parquet:", os.path.getsize("bench.parquet")/1e6, "MB")

t0=time.time(); pd.read_csv("bench.csv", usecols=["gia_tri"]); t_csv=time.time()-t0
t0=time.time(); pd.read_parquet("bench.parquet", columns=["gia_tri"]); t_pq=time.time()-t0
print("Doc 1 cot - CSV:", round(t_csv,4), "s | Parquet:", round(t_pq,4), "s")
```

Lab 4 — kết quả mong đợi & giải thích

- Dung lượng: file .parquet thường chỉ bằng khoảng **15–30%** so với .CSV trên cùng dữ liệu này (nhờ nén theo cột + kiểu dữ liệu nhị phân thay vì văn bản)
- Tốc độ đọc 1 cột: Parquet thường nhanh hơn CSV **vài lần đến chục lần** — vì CSV phải đọc **toàn bộ** 5 cột rồi mới lọc, còn Parquet chỉ đọc đúng cột `gia_tri`
- Con số chính xác phụ thuộc máy chạy — hãy chạy lại trên máy của bạn và **so sánh với số liệu benchmark tham khảo** đã nêu ở phần lý thuyết (Parquet+Snappy $\approx$ 15–25% dung lượng CSV)

Lab 5 — Khám phá Transaction Log của Delta Lake

- Mở repo `TaiLieuThamKhao/P2_LuuTruPhanTan/delta-lake-examples/` (chính thức từ `delta.io`) — xem thư mục `notebooks/`
- Nếu có môi trường PySpark + Delta cài sẵn (theo hướng dẫn `README.md` của repo): chạy thử tạo một bảng Delta nhỏ, thực hiện lần lượt `INSERT`, `UPDATE`, `DELETE`, rồi mở thư mục `_delta_log/` để **xem trực tiếp** các file `0000000.json`, `0000001.json`...
- Nếu chưa có môi trường Spark: đọc **nội dung một file JSON commit mẫu** trong tài liệu chính thức và đối chiếu với sơ đồ đã học ở lý thuyết — xác định phần nào ghi “add file”, phần nào ghi “remove file”

Lab 5 — minh họa cấu trúc 1 file commit JSON (rút gọn)

```
// ví dụ nội dung 000001.json (sau 1 lệnh UPDATE)
{"commitInfo": {"timestamp": 17500000001000,
  "operation": "UPDATE", "operationMetrics": {
    "numUpdatedRows": "120"}}}
{"remove": {"path": "part-0001.snappy.parquet",
  "deletionTimestamp": 17500000001000}}
{"add": {"path": "part-0007.snappy.parquet",
  "size": 843211, "dataChange": true}}
// remove + add cùng tồn tại trong 1 commit
// => file cũ KHÔNG bị sửa, chỉ bị "danh dấu bỏ"
// và thay bằng file mới -- đúng nguyên tắc immutable
```

- Vì sao commit UPDATE lại tạo ra một cặp **remove + add** thay vì **sửa trực tiếp** file Parquet cũ? Liên hệ với tính chất “immutable” của block trong HDFS/GFS đã học ở đầu buổi
- Nếu muốn **time travel** về trạng thái bảng **trước** lệnh UPDATE này, engine cần làm gì với 2 file JSON 000000.json và 000001.json?

Lab 6 — Bài tập điền bảng: chọn Table Format phù hợp

- Với mỗi tình huống dưới đây, chọn **Delta Lake** / **Iceberg** / **Hudi** và giải thích ngắn gọn lý do (liên hệ đặc điểm đã học ở lý thuyết)

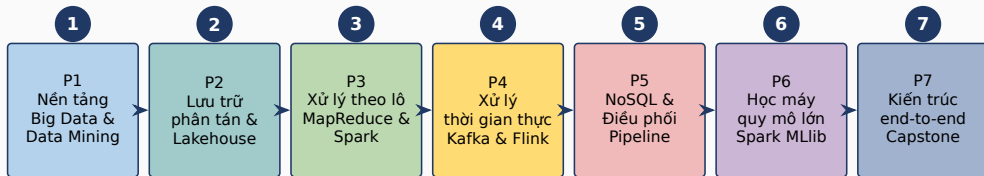
Lab 6 — tình huống cần chọn table format

Tình huống	Đặc điểm workload
A	Cập nhật vị trí hàng triệu thiết bị IoT mỗi vài giây, cần upsert liên tục
B	Đội ngũ dùng cả Spark, Trino và Flink cùng đọc chung một bảng, cần đa engine
C	Đã dùng Databricks toàn bộ hạ tầng, cần tích hợp sâu, ưu tiên đơn giản vận hành

- Tình huống A → **Apache Hudi** — thế mạnh cốt lõi là upsert/near real-time đã học
- Tình huống B → **Apache Iceberg** — thiết kế đa engine, hidden partitioning giúp mọi engine truy vấn nhất quán mà không cần biết chi tiết partition
- Tình huống C → **Delta Lake** — tích hợp sâu nhất với hệ sinh thái Databricks/Spark, vận hành đơn giản trong bối cảnh đó

BÀI TẬP THIẾT KẾ NHÓM

Mở rộng case study nền tảng gọi xe & giao đồ ăn đã phân tích ở phần lý thuyết



- Công ty case study (đã học ở lý thuyết) muốn mở rộng thêm một luồng dữ liệu mới: **đánh giá/review** của khách hàng sau mỗi chuyến đi (văn bản tự do + điểm sao 1–5)
- Yêu cầu: dữ liệu này sẽ được dùng để (1) hiển thị trên app, (2) phân tích cảm xúc (sentiment analysis) bằng ML ở Phần 6
- Nhiệm vụ nhóm: xác định — lớp Bronze/Silver/Gold sẽ tổ chức dữ liệu này ra sao? Định dạng file nào? Table format nào (nếu cần)? Chiến lược partition? Có cần Erasure Coding hay cross-region replication không?

Bài tập thiết kế — gợi ý hướng phân tích

- Đặc điểm dữ liệu review: khối lượng **nhỏ hơn nhiều** so với GPS ping (mỗi chuyến đi chỉ 1 review), **ít khi bị sửa** sau khi gửi — khác hẳn đặc điểm “upsert liên tục” của bảng GPS
- ⇒ Gợi ý: không cần Hudi cho trường hợp này — Delta Lake hoặc thậm chí Parquet thuần (không cần ACID phức tạp) đã đủ, vì tần suất update rất thấp
- Với văn bản tự do (review), cân nhắc: có nên lưu **nguyên văn** ở Silver để tái sử dụng cho các mô hình NLP tương lai (chưa biết trước), hay chỉ lưu **đặc trưng đã trích xuất** (feature) ở Gold? → Đây chính là lý do lớp Bronze/Silver luôn giữ dữ liệu **gần nguyên bản nhất có thể**

- Viết lại Lab 4 (so sánh CSV vs Parquet) nhưng với dữ liệu **20 cột** thay vì 5 cột — quan sát mức tiết kiệm I/O khi chỉ đọc 2 cột có tăng lên rõ rệt hơn không? So sánh với ví dụ số đã tính ở lý thuyết (tiết kiệm 10 lần với 20 cột)
- Đọc README.md của repo `iceberg-spark-docker-quickstart` đã tải về — liệt kê các bước cần làm để khởi động một môi trường Iceberg + Spark cục bộ bằng Docker (không bắt buộc chạy thật nếu máy không đủ tài nguyên)
- Tính lại bài toán Lab 1 với dữ liệu = 2PB thay vì 500TB, và thử với replication factor = 2 thay vì 3 — nhận xét về đánh đổi độ tin cậy

Checklist ôn tập Phần 2

- ☐ Giải thích được vai trò NameNode/DataNode và Master/Chunkserver, chỉ ra điểm tương đồng
- ☐ Tính được dung lượng vật lý cần thiết với một replication factor cho trước
- ☐ Phân biệt row-based và columnar storage, giải thích column pruning & predicate pushdown
- ☐ Vẽ được sơ đồ phân lớp Bronze/Silver/Gold và giải thích vai trò từng lớp
- ☐ Phát biểu được định lý CAP và áp dụng vào phân loại một hệ thống cụ thể
- ☐ So sánh được Delta Lake, Iceberg, Hudi và chọn đúng công cụ theo tình huống

Chuẩn bị cho Tuần 4 — Xử lý theo lô: MapReduce & Spark

- Tuần 4 sẽ học **cách xử lý** dữ liệu đã lưu trên HDFS/Lakehouse mà ta vừa thiết kế ở Tuần 3 — MapReduce (mô hình lập trình gốc) và Apache Spark (thế hệ kế tiếp, nhanh hơn)
- Nên cài sẵn: Java JDK (cho bài tập MapReduce), Python + PySpark (cho bài tập Spark) — xem hướng dẫn cài đặt trong `README.md` của các repo đã tải
- Ôn lại: nguyên lý “di chuyển tính toán rẻ hơn di chuyển dữ liệu” đã học ở Tuần 3 — đây chính là ý tưởng cốt lõi của MapReduce

Hỏi & Đáp

Câu hỏi về HDFS, GFS, định dạng file, hay Lakehouse?